

DATA STRUCTURE

Introduction

Algorithm:

An algorithm may be defined as a finite sequence of instructions each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time.

The algorithm word originated from the Arabic word “Algorism” which is linked to the name of the Arabic mathematician Al Khwarizmi. He is considered to be the first algorithm designer for adding numbers.

Structure and Properties of Algorithm:

An algorithm has the following structure

- Input Step
- Assignment Step
- Decision Step
- Repetitive Step
- Output Step

An algorithm will be good with the following properties:

Finiteness: An algorithm must terminate after a finite number of steps.

Definiteness: The steps of the algorithm must be precisely defined or unambiguously specified.

Generality: An algorithm must be generic enough to solve all problems of a particular class.

Effectiveness: the operations of the algorithm must be basic enough to be put down on pencil and paper. They should not be too complex to warrant writing another algorithm for the operation.

Input-Output: The algorithm must have certain initial and precise inputs, and outputs that may be generated both at its intermediate and final steps.

An algorithm does not enforce a language or mode for its expression but only demands adherence to its properties.

Practical Algorithm Design Issues:

DATA STRUCTURE

To save time (Time Complexity): A program that runs faster is a better program.

To save space (Space Complexity): A program that saves space over a competing program is considerable desirable.

Introduction to Data structures

In computer terms, a data structure is a Specific way to store and organize data in a computer's memory so that these data can be used efficiently later. Data may be arranged in many different ways such as the logical or mathematical model for a particular organization of data is termed as a data structure. The variety of a particular data model depends on the two factors –

- Firstly, it must be loaded enough in structure to reflect the actual relationships of the data with the real world object.
- Secondly, the formation should be simple enough so that anyone can efficiently process the data each time it is necessary.

Categories of Data Structure:

The data structure can be sub divided into major types:

- Linear Data Structure
- Non-linear Data Structure

Linear Data Structure:

A data structure is said to be linear if its elements combine to form any specific order. There are basically two techniques of representing such linear structure within memory.

- First way is to provide the linear relationships among all the elements represented by means of linear memory location. These linear structures are termed as arrays
- The second technique is to provide the linear relationship among all the elements represented by using the concept of pointers or links. These linear structures are termed as linked lists. The common examples of linear data structure are:

- Arrays
- Queues
- Stacks

DATA STRUCTURE

- Linked lists

Nonlinear Data Structure:

This structure is mostly used for representing data that contains a hierarchical relationship among various elements. Examples of Non Linear Data Structures are listed below:

- Graphs
- Family of trees and
- Table of contents Tree:

In this case, data often contain a hierarchical relationship among various elements. The data structure that reflects this relationship is termed as rooted tree graph or a tree.

Graph: In this case, data sometimes hold a relationship between the pairs of elements which is not necessarily following the hierarchical structure. Such data structure is termed as a Graph.

Operations on Data Structures

The basic operations that are performed on data structures are as follows:

Insertion: Insertion means addition of a new data element in a data structure.

Deletion: Deletion means removal of a data element from a data structure if it is found.

Searching: Searching involves searching for the specified data element in a data structure.

Traversal: Traversal of a data structure means processing all the data elements present in it.

Sorting: Arranging data elements of a data structure in a specified order is called sorting.

Merging: Combining elements of two similar data structures to form a new data structure of the same type, is called merging.

DATA STRUCTURE

Algorithm complexity

Efficiency of Algorithms:

The performances of algorithms can be measured on the scales of time and space. The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical and the other is experimental. In performance analysis we use analytical methods, while in performance measurement we conduct experiments.

Time Complexity: The time complexity of an algorithm or a program is a function of the running time of the algorithm or a program. In other words, it is the amount of computer time needs to run to completion.

Space Complexity: The space complexity of an algorithm or program is a function of the space needed by the algorithm or program to run to completion.

The time complexity of an algorithm can be computed either by an empirical or theoretical approach. The empirical or posteriori testing approach calls for implementing the complete algorithms and executing them on a computer for various instances of the problem. The algorithm whose implementation yields the least time is consider as the best among the candidate algorithmic solutions.

Analyzing Algorithms:

Suppose M is an algorithm, and suppose n is the size of the input data. Clearly the complexity $f(n)$ of M increases as n increases. It is usually the rate of increase of $f(n)$ with some standard functions. The most common computing times are

$O(1)$, $O(\log_2 n)$, $O(n)$, $O(n \log_2 n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$

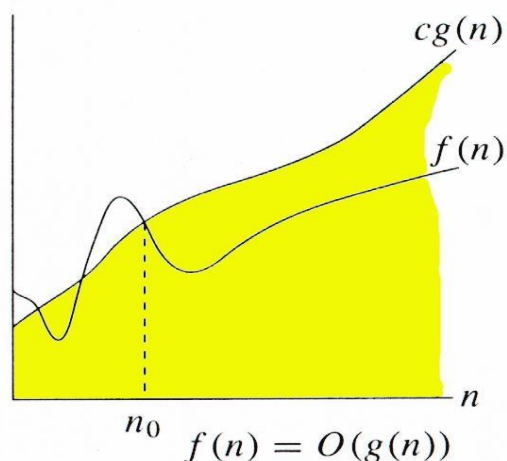
Asymptotic Notations:

It is often used to describe how the size of the input data affects an algorithm's usage of computational resources. Running time of an algorithm is describe as a function of input size n for large n .

Big oh(O): Definition: $f(n) = O(g(n))$ (read as f of n is big oh of g of n) if there exist a positive integer n_0 and a positive number c such that $|f(n)| \leq c|g(n)|$ for all $n \geq n_0$. Here $g(n)$ is the upper bound of the function $f(n)$.

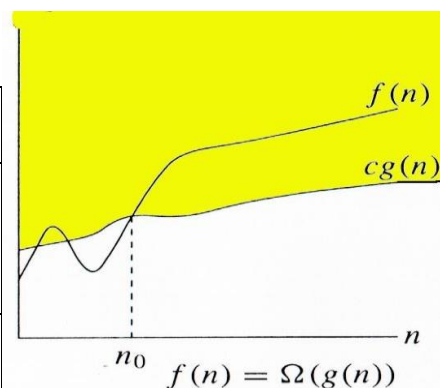
DATA STRUCTURE

$f(n)$	$g(n)$	
$3 \quad 2$ $16n + 45n$ $+$ $12n$	3 n	$3 f(n) = O(n)$
$34n - 40$	n	$f(n) = O(n)$
50	1	$f(n) = O(1)$



Omega(Ω): Definition: $f(n) = \Omega(g(n))$ (read as f of n is omega of g of n), if there exists a positive integer n_0 and a positive number c such that $|f(n)| \geq c |g(n)|$ for all $n \geq n_0$. Here $g(n)$ is the lower bound of the function $f(n)$.

$f(n)$	$g(n)$	
$3 \quad 2$ $16n + 8n + 2$	3 n	$3 f(n) = \Omega(n)$ $)$
$24n + 9$	n	$f(n) = \Omega(n)$



Theta(Θ): Definition: $f(n) = \Theta(g(n))$ (read as f of n is theta of g of n), if there exists a positive integer n_0 and two positive constants c_1 and c_2 such that $c_1 |g(n)| \leq |f(n)| \leq c_2 |g(n)|$ for all $n \geq n_0$. The function $g(n)$ is both an upper bound and a lower bound for the function $f(n)$ for all values of n , $n \geq n_0$.

DATA STRUCTURE

$f(n)$	$g(n)$	
$16n^3 + 30n^2 - 90$	n^2	$f(n) = \Theta(n^2)$
$7 \cdot 2^n + 30n$	2^n	$f(n) = \Theta(2^n)$

